ATMEGA4809 Breadboard Kit

CHIP: *ATMEGA4809*	**Parts**: *18 (+1 optional programmer)*

Kit Description:

This ATMEGA4809 Developer Kit & Datasheet has everything you need to build your very own MCU on a breadboard/protoboard. This kit contains one ATMEGA4809, capacitors, resistors, led's, LDO voltage regulator, ESD protections, memory, a programmable timer, switches & optionally an RP2040 for seamless UART programming. This kit's purpose is not only for education however, it is truly a fantastic applied microcontroller. Featuring an 8 bit 8mhz (up to 20mhz depending on bootloader & external crystal) AVR processor. This ATMEGA is tested, reliable and a foundation for any hobbyist/engineer. If you choose to include the RP2040 as the programmer, you get built-in USB-C > UART programming and a serial monitor for all your debugging needs. You may also opt for the classic USB to serial adapter, featuring chips such as the CH340.

This ATMEGA is pre-burned with a bootloader and typically operates on a 8mhz internal oscillator. If you wish, reburning a bootloader is easy and unlocks more possibilities.

Happy building!

ATMEGA4809 PINOUT:

PIN	NAME	DOES THIS	PIN	NAME	DOES THIS
1	PC0	General IO, TX	21	PE2	General IO, AIN10, SCK
2	PC1	General IO, RX	22	PE3	General IO, AIN11, SS
3	PC2	General IO, I2C	23	PF0	TX
4	PC3	General IO, I2C	24	PF1	RX
5	VDD	+3.3v (Can be +5v)	25	PF2	General IO, AIN12, I2C
6	GND	Ground	26	PF3	General IO, AIN13, I2C
7	PC4	General IO, TX, PWM	27	PF4	General IO, AIN14, TX
8	PC5	General IO, RX, PWM	28	PF5	General IO, AIN15, RX

PIN	NAME	DOES THIS	PIN	NAME	DOES THIS
9	PD0	General IO, AIN0, PWM	29	PF6 (Reset)	General IO, RESET
10	PD1	General IO, AIN1, AC0	30	UPDI	UPDI Programming
11	PD2	General IO, AIN2, AC0	31	VDD	+3.3v (Can be +5v)
12	PD3	General IO, AIN3, AC0	32	GND	Ground
13	PD4	General IO, AIN4, AC0	33	PA0	EXTCLK, TX, WO0
14	PD5	General IO, AIN5, AC0	34	PA1	RX, WO1
15	PD6	General IO, AIN6, AC0	35	PA2	General IO, I2C, WO2
16	PD7	General IO, AIN7, VREFA	36	PA3	General IO, I2C, WO3
17	AVDD	+3.3v (Can be +5v)	37	PA4	General IO, MOSI, TX
18	GND	Ground	38	PA5	General IO, I2C, MISO, RX
19	PE0	General IO,MOSI, AIN8	39	PA6	General IO, I2C, SCK
20	PE1	General IO, MISO, AIN9	40	PA7	General IO, SS

Programming:

There are 3 common ways to program the ATMEGA

1. The typical method of programming the ATMEGA4809 uses a USB to serial/UART/UPDI adapter, such as the CH340 found on Amazon. If using UART, you connect the adapter RX to the ATMEGA UPDI, and TX to UPDI through a 4.7k ohm resistor. Needing only 3 connections, UPDI, GND and VDD, it is a dead simple way to program the ATMEGA and access the serial monitor.
2. A secondary popular method to program the ATMEGA is using another arduino MCU, such as the arduino uno. Attached [Here] is code for ArduinoISP, which after uploaded to the ATMEGA, allows the user to upload minicore boards via SPI (uploaded via programmer). Be sure to program minicore with correct settings (8mhz internal by default). For reading serial monitor, simply upload [This] code to the arduino uno after

programming it, and connect selected UART lines to UPDI, giving TX a 47k ohm series resistor.

3. A third method of programming is using the RP2040 optionally included in the kit. Working on a basic pass through UART program found [Here], you may again use minicore, and the bootloader settings (internal 8mhz oscillator), and upload regularly. Once again, be sure to give TX a 4.7k ohm series resistor, and RX a straight UPDI connection. You must power the ATMEGA via 3.3v if using RP2040. Be sure to pull reset low right as programming starts. You may use a pushbutton to GND or a GPIO, be sure to also include 10k pull up to VDD on reset. Right as programming starts in arduinoIDE, send a short quick pulse to reset the ATMEGA.

Parts in kit:

COMPONENT	# Included	JOB
ATMEGA4809	1	Microcontroller
3.3v 0.8A LDO	1	Voltage Regulator (3.3v)
NE555P (Timer)	1	Programmable Timer & Clock
25AA320A (EEPROM)	1	Non Volatile SPI Memory
100nF Ceramic Capacitor	8	Local Decoupling
1uF Ceramic Capacitor	2	Local Decoupling
47uF Electrolytic Capacitor	1	Bulk Decoupling
100uF Electrolytic Capacitor	2	Bulk Decoupling
0.5A Polyfuse	1	VDD Overcurrent Protection
1A Schottky Diode	1	VDD ESD Protection
Ferrite Bead	1	Analog VDD Filtering
33 Ohm Resistor	5	Series resistors for high speeds
470 Ohm Resistor	3	LED resistors
1k Ohm Resistor	3	Pull up/down
4.7k Ohm Resistor	5	Pull up/down
10k Ohm Resistor	5	Pull up/down

COMPONENT	# Included	JOB
Green LED	3	Power good, general use, IO's
Tactile Switch	2	Reset/General Purpose
RP2040 Programmer (Optional)	1	UART Programming with USB-C

Example Layout:

1. Power Management

Place the MCU on the center of the breadboard, giving each pin its own row. Connect VDD to your power rail, and AVCC through a ferrite bead to your power rail. Be sure to also connect all ground pins to your ground rail. On both VDD and AVCC pins, place 100nF capacitors from power to the ground rail. On the VDD rail, place a 47uF electrolytic capacitor, being mindful of polarity, along with a 1uF ceramic capacitor. If using a 6v power supply (batteries), be sure to use the LDO to regulate it down to 3.3v. If using the RP2040 programmer, you should use 3.3v as your power supply. Optionally, you can choose to use a polyfuse in series, and the diode to ground as ESD protection.

2. Programming

Connect PF6 to a tactile switch, which is connected to ground on the other side, also provide PF6 a 10k pull up to VDD. Connect the programmer UPDI pin to the ATMega UPDI pin and ensure common grounds.

3. Application

To use the code featured in the example code section of this datasheet, connect 1 side of a tactile button to GND and the other side to PD2

That's it! The basic example layout is surprisingly simple.

Happy Building!

Example Code (MCU + Button):

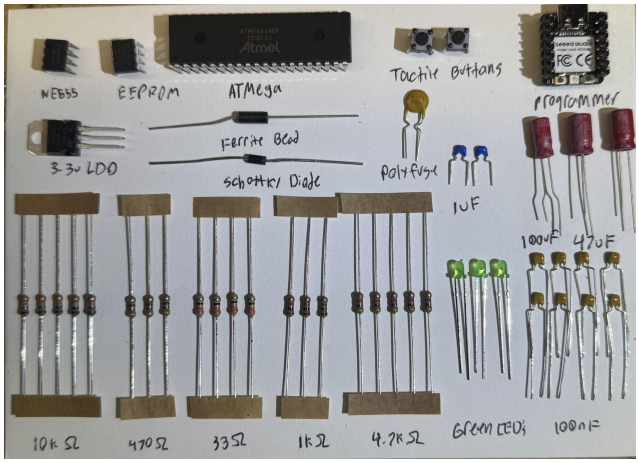
```
/*
 * ATMEGA4809 Dev Kit -- Example Sketch: Button Press Counter
 * ----------------------------------------------------------
 * Press the tactile button. Each press prints the press number
 * and the elapsed time (in ms) since the board booted.
 *
 * WIRING:
 *   Tactile Button -> D2, other side to GND (uses internal pull-up)
 */

const uint8_t BUTTON_PIN = PIN_PD2;
```

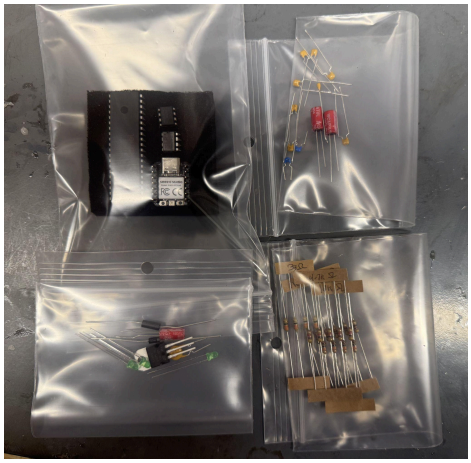
```
int pressCount = 0;
bool lastState = HIGH;

void setup() {
  Serial.begin(9600);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
}

void loop() {
  bool currentState = digitalRead(BUTTON_PIN);

  if (lastState == HIGH && currentState == LOW) {
    pressCount++;
    Serial.print("Press #");
    Serial.print(pressCount);
    Serial.print(" - Elapsed time: ");
    Serial.print(millis());
    Serial.println(" ms");
    delay(50); // basic debounce
  }

  lastState = currentState;
}
```

Applications:

- Embedded systems prototyping & education
- IoT sensor nodes
- Robotics & motor control
- Data logging
- Custom Arduino-alternative projects
- Home automation

PHOTOS:





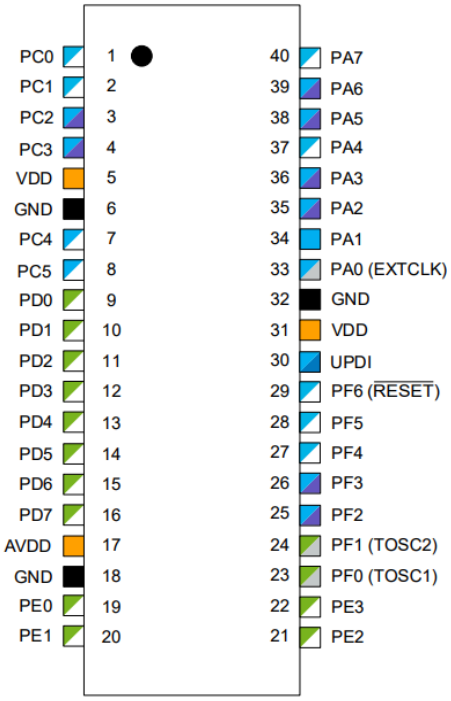
Datasheets & Pinout Images:

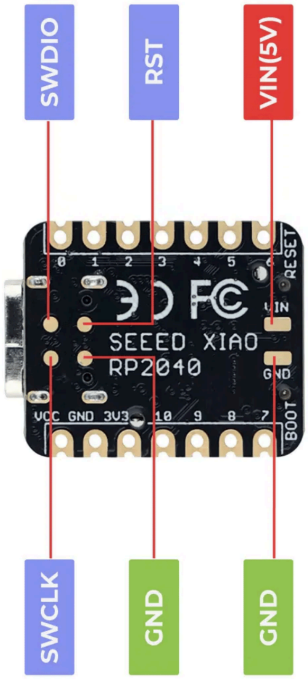
[ATMEGA4809 Datasheet]

[NE555 Datasheet]

[25AA320A Datasheet]

[RP2040 Datasheet]





On the Left: ATMEGA4809
On the Right: Xiao RP2040

Notes and Debugging:

- By default the ATMEGA will operate on 8mhz internal at 3.3v. This can be changed depending on the programming method.
- ATMEGA comes preburned with bootloader
- If uploading via Arduino UNO, an ATMEGA program can only be flashed when the UNO has the arduinoISP loaded.
- BAUD Rate typically set at 9600
- Be sure to connect VDD & AVCC for all ATMEGA use
- Install correct MiniCore library found [Here]
- Be persistent, working with bare microcontroller chips is NOT easy, but it WILL work. The ATMEGA is incredibly resilient and you are too. Direct any questions to Meridian Electronics, were happy to help